

Projekt Bazy Danych

Role Użytkowników:

- **Administrator:** Posiada pełny dostęp do wszystkich funkcji systemu, w tym zarządzania użytkownikami, konfiguracji systemu, ofertami, rezerwacjami i raportami.
- **Manager:** Posiada uprawnienia do zarządzania ofertami, rezerwacjami, klientami oraz generowania podstawowych raportów.
- **Pracownik:** Ograniczony dostęp — może przeglądać oferty, przyjmować rezerwacje, potwierdzać i anulować rezerwacje, przeglądać dane klientów oraz generować rachunki.
- **Klient:** Posiada uprawnienia do przeglądania ofert, składania rezerwacji, zarządzania swoimi danymi i przeglądania historii rezerwacji.

Funkcje Systemu i Uprawnienia:

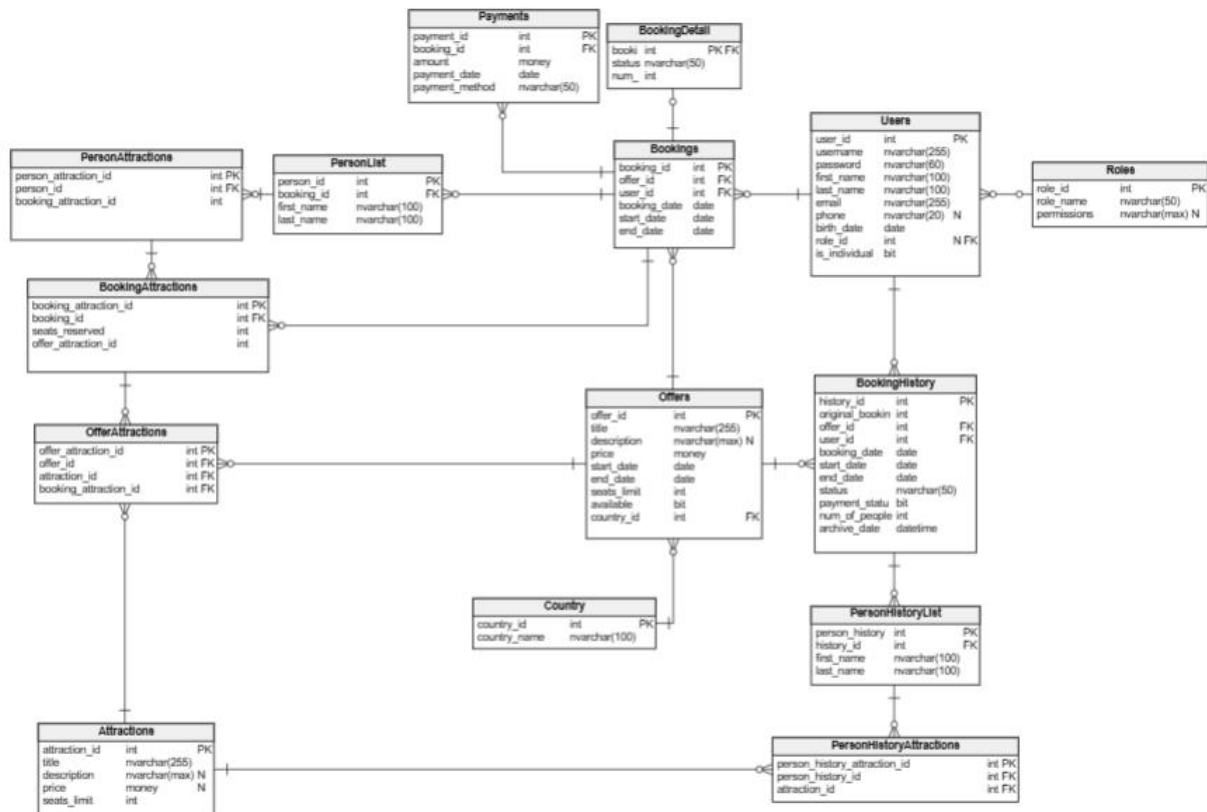
Funkcja	Administrator	Manager	Pracownik	Klient
Zarządzanie Ofertami Turystycznymi				
Dodawanie oferty (wycieczki, wyjazdy)	✓	✓		
Edycja danych oferty	✓	✓		
Usuwanie oferty	✓	✓		
Wyświetlanie listy dostępnych ofert	✓	✓	✓	✓
Zarządzanie Atrakcjami				
Dodawanie/edycja atrakcji	✓	✓		
Zarządzanie Rezerwacjami				
Składanie nowej rezerwacji	✓	✓	✓	✓
Potwierdzanie rezerwacji	✓	✓	✓	
Anulowanie rezerwacji	✓	✓	✓	✓

Edycja danych rezerwacji	✓	✓	✓	tylko swoje
Przeglądanie listy rezerwacji	✓	✓	✓	tylko swoje
Przeglądanie szczegółów rezerwacji klienta	✓	✓	✓	tylko swoje
Zarządzanie Klientami				
Dodawanie nowego klienta	✓	✓	✓	
Edycja danych klienta	✓	✓	✓	tylko swoje
Przeglądanie listy klientów	✓	✓	✓	
Przeglądanie historii rezerwacji klienta	✓	✓	✓	tylko swoje
Zarządzanie Użytkownikami Systemu (Pracownicy)				
Dodawanie nowego pracownika	✓			
Edycja danych pracownika	✓			
Usuwanie pracownika	✓			
Raporty i Statystyki				
Generowanie raportów sprzedaży ofert	✓	✓		
Generowanie raportów popularności destynacji	✓	✓		
Generowanie raportów o klientach	✓	✓		
Płatności i Rozliczenia				

Oznaczanie rezerwacji jako zapłacone	✓	✓	✓	
Generowanie rachunków / faktur	✓	✓	✓	tylko własne
Wycena rezerwacji / oferty	✓	✓		
Inne Funkcje				
Edycja systemu powiadomień	✓	✓		

Raport dotyczący struktury i integralności bazy danych systemu rezerwacyjnego

1. Schemat bazy danych



2. Opis szczegółowy tabel i atrybutów

Tabela: Attractions

Opis: Dane o atrakcjach turystycznych.

Atrybut	Typ danych	Opis
attraction_id	int	Identyfikator atrakcji (PK)
title	nvarchar(255)	Nazwa atrakcji

description	nvarchar(max)	Opis atrakcji
price	money	Cena
seats_limit	int	Limit miejsc

Warunki integralności:

PK: attraction_id

CHECK: price >= 0, seats_limit > 0

Kod DDL:

```
CREATE TABLE Attractions (
    attraction_id int NOT NULL IDENTITY(1, 1),
    title nvarchar(255) NOT NULL,
    description nvarchar(max) NULL,
    price money NULL,
    seats_limit int NOT NULL,
    CONSTRAINT CHECK_Attractions_Price CHECK (( price >= 0 )),
    CONSTRAINT CHECK_Attractions_Seats CHECK (( seats_limit > 0 )),
    CONSTRAINT Attractions_pk PRIMARY KEY (attraction_id)
);
```

Tabela: BookingAttractions

Opis: Rezerwacje atrakcji powiązane z ofertami.

Atrybut	Typ danych	Opis
booking_attraction_id	int	Identyfikator rezerwacji (PK)

booking_id	int	Rezerwacja (FK)
person_attraction_id	int	Atrakcji oferty osób (FK)
seats_reserved	int	Liczba zarezerwowanych miejsc
offer_attraction_id	int	Oferta atrakcji

Warunki integralności:

PK: booking_attraction_id

CHECK: seats_reserved > 0

FK: booking_id → Bookings

FK: offer_attraction_id → OfferAttractions

Kod DDL:

```
CREATE TABLE u_mjopek.dbo.BookingAttractions (
    booking_attraction_id int IDENTITY(1,1) NOT NULL,
    booking_id int NOT NULL,
    seats_reserved int NOT NULL,
    offer_attraction_id int NULL,
    CONSTRAINT BookingAttractions_pk PRIMARY KEY (booking_attraction_id)
);
```

Tabela: BookingDetail

Opis: Szczegóły rezerwacji.

Atrybut	Typ danych	Opis
booking_id	int	Identyfikator rezerwacji (PK)

status	nvarchar(50)	Status rezerwacji
num_of_people	int	Liczba osób

Warunki integralności:

PK: booking_id

CHECK: status IN ('pending', 'confirmed', 'cancelled'), num_of_people > 0

FK: booking_id → Bookings

Kod DDL:

```
CREATE TABLE BookingDetail (
    booking_id int NOT NULL,
    status nvarchar(50) NOT NULL,
    num_of_people int NOT NULL,
    CONSTRAINT CHECK_BookingDetail_Status CHECK (( status IN ( 'pending' , 'confirmed' , 'cancelled' ) )),
    CONSTRAINT CHECK_BookingDetail_NumOfPeople CHECK (( num_of_people > 0 )),
    CONSTRAINT PK_BookingDetail PRIMARY KEY (booking_id)
);
```

Tabela: BookingHistory

Opis: Archiwum rezerwacji użytkowników.

Atrybut	Typ danych	Opis
history_id	int	Identyfikator historii (PK)
original_booking_id	int	Identyfikator oryginalnej rezerwacji

offer_id	int	Oferta (FK)
user_id	int	Użytkownik (FK)
booking_date	date	Data rezerwacji
start_date	date	Data rozpoczęcia
end_date	date	Data zakończenia
status	nvarchar(50)	Status rezerwacji
payment_status	bit	Status płatności
num_of_people	int	Liczba osób
archive_date	datetime	Data archiwizacji (domyślnie teraz)

Warunki integralności:

PK: history_id

DEFAULT: archive_date → GETDATE()

FK: offer_id → Offers

FK: user_id → Users

Kod DDL:

```
CREATE TABLE BookingHistory (
    history_id int NOT NULL IDENTITY(1, 1),
    original_booking_id int NOT NULL,
    offer_id int NOT NULL,
    user_id int NOT NULL,
```

```
booking_date date NOT NULL,  
start_date date NOT NULL,  
end_date date NOT NULL,  
status nvarchar(50) NOT NULL,  
payment_status bit NOT NULL,  
num_of_people int NOT NULL,  
  
archive_date datetime NOT NULL CONSTRAINT DF_BookingHistory_ArchiveDate DEFAULT  
(getdate ( )),  
  
CONSTRAINT BookingHistory_pk PRIMARY KEY (history_id)  
);
```

Tabela: Bookings

Opis: Rezerwacje użytkowników.

Atrybut	Typ danych	Opis
booking_id	int	Identyfikator rezerwacji (PK)
offer_id	int	Oferta (FK)
user_id	int	Użytkownik (FK)
booking_date	date	Data rezerwacji (domyślnie teraz)
start_date	date	Data rozpoczęcia
end_date	date	Data zakończenia

Warunki integralności:

PK: booking_id

DEFAULT: booking_date → GETDATE()

CHECK: end_date >= start_date

FK: offer_id → Offers

FK: user_id → Users

Kod DDL:

```
CREATE TABLE Bookings (  
    booking_id int NOT NULL IDENTITY(1, 1),  
    offer_id int NOT NULL,  
    user_id int NOT NULL,  
    booking_date date NOT NULL CONSTRAINT DF_Bookings_BookingDate DEFAULT (getdate  
( )),  
    start_date date NOT NULL,  
    end_date date NOT NULL,  
    CONSTRAINT CHECK_Bookings_Dates CHECK (( ( end_date >= start_date ) )),  
    CONSTRAINT Bookings_pk PRIMARY KEY (booking_id)  
);
```

Tabela: Country

Opis: Lista krajów.

Atrybut	Typ danych	Opis
country_id	int	Identyfikator kraju (PK)
country_name	nvarchar(100)	Nazwa kraju (unikalna)

Warunki integralności:

PK: country_id

UNIQUE: country_name

Kod DDL:

```
CREATE TABLE Country (  
    country_id int NOT NULL IDENTITY(1, 1),  
    country_name nvarchar(100) NOT NULL,  
    CONSTRAINT AK_Country_Name UNIQUE (country_name),  
    CONSTRAINT Country_pk PRIMARY KEY (country_id)  
);
```

Tabela: OfferAttractions

Opis: Atrakcje powiązane z ofertami.

Atrybut	Typ danych	Opis
offer_attraction_id	int	Identyfikator (PK)
offer_id	int	Oferta (FK)
attraction_id	int	Atrakcja (FK)
booking_attraction_id	int	Identyfikator zarezerwowanej atrakcji (FK)

Warunki integralności:

PK: offer_attraction_id

FK: offer_id → Offers

FK: attraction_id → Attractions

FK: booking_attraction_id → BookingAttraction

Kod DDL:

CREATE TABLE OfferAttractions (

offer_attraction_id int NOT NULL IDENTITY(1, 1),

offer_id int NOT NULL,

attraction_id int NOT NULL,

CONSTRAINT OfferAttractions_pk PRIMARY KEY (offer_attraction_id)

);

Tabela: Offers

Opis: Oferty turystyczne.

Atrybut	Typ danych	Opis
offer_id	int	Identyfikator oferty (PK)
title	nvarchar(255)	Nazwa oferty (unikalna)
description	nvarchar(max)	Opis
price	money	Cena
start_date	date	Data rozpoczęcia
end_date	date	Data zakończenia

seats_limit	int	Limit miejsc
available	bit	Dostępność (domyślnie 1)
country_id	int	Kraj (FK)

Warunki integralności:

PK: offer_id

UNIQUE: title

DEFAULT: available → 1

CHECK: price >= 0, seats_limit > 0, end_date >= start_date and start_date >= CAST(GETDATE())

FK: country_id → Country

Kod DDL:

```
CREATE TABLE Offers (
    offer_id int NOT NULL IDENTITY(1, 1),
    title nvarchar(255) NOT NULL,
    description nvarchar(max) NULL,
    price money NOT NULL,
    start_date date NOT NULL,
    end_date date NOT NULL,
    seats_limit int NOT NULL,
    available bit NOT NULL CONSTRAINT DF_Offers_Available DEFAULT (1),
    country_id int NOT NULL,
    CONSTRAINT AK_Offers_Title UNIQUE (title),
    CONSTRAINT CHECK_Offers_Price CHECK (price >= 0),
    CONSTRAINT CHECK_Offers_Dates CHECK (end_date >= start_date),
    CONSTRAINT CHECK_Offers_Seats CHECK (seats_limit > 0),
    CONSTRAINT CHECK_Offers_Start_Date CHECK (start_date >= CAST(GETDATE() AS
DATE)),
```

CONSTRAINT Offers_pk PRIMARY KEY (offer_id)

);

Tabela: Payments

Opis: Informacje o płatnościach.

Atrybut	Typ danych	Opis
payment_id	int	Identyfikator płatności (PK)
booking_id	int	Rezerwacja (FK)
amount	money	Kwota
payment_date	date	Data płatności (domyślnie teraz)
payment_method	nvarchar(50)	Metoda płatności

Warunki integralności:

PK: payment_id

DEFAULT: payment_date → GETDATE()

CHECK: amount >= 0, payment_method IN ('credit card', 'bank transfer', 'cash')

FK: booking_id → Bookings

Kod DDL:

```
CREATE TABLE Payments (
```

```
    payment_id int NOT NULL IDENTITY(1, 1),
```

```
    booking_id int NOT NULL,
```

```
    amount money NOT NULL,
```

```

    payment_date date NOT NULL CONSTRAINT DF_Payments_PaymentDate DEFAULT (getdate
( )),

    payment_method nvarchar(50) NOT NULL,

    CONSTRAINT CHECK_Payments_Amount CHECK (( ( amount >= 0 ) )),

    CONSTRAINT CHECK_Payments_Method CHECK (( ( payment_method IN ( 'credit card' , 'bank
transfer' , 'cash' ) ) )),

    CONSTRAINT Payments_pk PRIMARY KEY (payment_id)

);

```

Tabela: PersonAttractions

Opis: Osoby przypisane do rezerwacji atrakcji.

Atrybut	Typ danych	Opis
person_attraction_id	int	Identyfikator (PK)
person_id	int	Osoba (FK)
booking_id	int	Rezerwacja(FK)

Warunki integralności:

PK: person_attraction_id

FK: person_id → PersonList

FK: booking_attraction_id → BookingAttractions

Kod DDL:

```

CREATE TABLE u_mzes.dbo.PersonAttractions (

    person_attraction_id int IDENTITY(1,1) NOT NULL,

    person_id int NOT NULL,

    booking_attraction_id int NOT NULL,

```

CONSTRAINT PersonAttractions_pk PRIMARY KEY (person_attraction_id)

);

Tabela: PersonHistoryAttractions

Opis: Osoby przypisane do archiwalnych atrakcji.

Atrybut	Typ danych	Opis
person_history_attraction_id	int	Identyfikator (PK)
person_history_id	int	Osoba w archiwum (FK)
attraction_id	int	Atrakcja (FK)

Warunki integralności:

PK: person_history_attraction_id

FK: person_history_id → PersonHistoryList

FK: attraction_id → Attractions

Kod DDL:

```
CREATE TABLE PersonHistoryAttractions (  
    person_history_attraction_id int NOT NULL IDENTITY(1, 1),  
    person_history_id int NOT NULL,  
    attraction_id int NOT NULL,  
    CONSTRAINT PersonHistoryAttractions_pk PRIMARY KEY (person_history_attraction_id)  
);
```

Tabela: PersonHistoryList

Opis: Lista osób w archiwalnych rezerwacjach.

Atrybut	Typ danych	Opis
person_history_id	int	Identyfikator (PK)
history_id	int	Rezerwacja w archiwum (FK)
first_name	nvarchar(100)	Imię
last_name	nvarchar(100)	Nazwisko

Warunki integralności:

PK: person_history_id

FK: history_id → BookingHistory

Kod DDL:

```
CREATE TABLE PersonHistoryList (  
    person_history_id int NOT NULL IDENTITY(1, 1),  
    history_id int NOT NULL,  
    first_name nvarchar(100) NOT NULL,  
    last_name nvarchar(100) NOT NULL,  
    CONSTRAINT PersonHistoryList_pk PRIMARY KEY (person_history_id)  
);
```

Tabela: PersonList

Opis: Lista osób przypisanych do rezerwacji.

Atrybut	Typ danych	Opis
person_id	int	Identyfikator (PK)
booking_id	int	Rezerwacja (FK)
first_name	nvarchar(100)	Imię
last_name	nvarchar(100)	Nazwisko

Warunki integralności:

PK: person_id

FK: booking_id → Bookings

Kod DDL:

```
CREATE TABLE PersonList (
    person_id int NOT NULL IDENTITY(1, 1),
    booking_id int NOT NULL,
    first_name nvarchar(100) NOT NULL,
    last_name nvarchar(100) NOT NULL,
    CONSTRAINT PersonList_pk PRIMARY KEY (person_id)
);
```

Tabela: Roles

Opis: Role i uprawnienia użytkowników.

Atrybut	Typ danych	Opis
role_id	int	Identyfikator roli (PK)
role_name	nvarchar(50)	Nazwa roli
permissions	nvarchar(max)	Uprawnienia (opcjonalnie)

Warunki integralności:

PK: role_id

Kod DDL:

```
CREATE TABLE Roles (  
    role_id int NOT NULL IDENTITY(1, 1),  
    role_name nvarchar(50) NOT NULL,  
    permissions nvarchar(max) NULL,  
    CONSTRAINT Roles_pk PRIMARY KEY (role_id)  
);
```

Tabela: Users

Opis: Dane zarejestrowanych użytkowników.

Atrybut	Typ danych	Opis
user_id	int	Identyfikator użytkownika (PK)

username	nvarchar(255)	Nazwa użytkownika
password	nvarchar(60)	Hasło
first_name	nvarchar(100)	Imię
last_name	nvarchar(100)	Nazwisko
email	nvarchar(255)	Adres e-mail
phone	nvarchar(20)	Numer telefonu
birth_date	date	Data urodzenia
role_id	int	Rola (FK)
is_individual	bit	Czy osoba indywidualna

Warunki integralności:

PK: user_id

UNIQUE: username, email

CHECK: DATEDIFF(YEAR, birth_date, GETDATE()) BETWEEN 18 AND 100

DEFAULT: is_individual → 1

FK: role_id → Roles

phone IS NULL OR phone NOT LIKE '%[^0-9]%'

email LIKE '%@%'

Kod DDL:

```

CREATE TABLE Users (
    user_id int NOT NULL IDENTITY(1, 1),
    username nvarchar(255) NOT NULL,
    password nvarchar(60) NOT NULL,
    first_name nvarchar(100) NOT NULL,
    last_name nvarchar(100) NOT NULL,
    email nvarchar(255) NOT NULL,
    phone nvarchar(20) NULL,
    birth_date date NOT NULL,
    role_id int NULL,
    is_individual bit NOT NULL CONSTRAINT DF_Users_IsIndividual DEFAULT (1),

    CONSTRAINT Users_pk PRIMARY KEY (user_id),
    CONSTRAINT AK_Users_Username UNIQUE (username),
    CONSTRAINT AK_Users_Email UNIQUE (email),

    CONSTRAINT CK_Users_Age CHECK (
        DATEDIFF(YEAR, birth_date, GETDATE()) BETWEEN 18 AND 100
    ),
    CONSTRAINT CK_Users_Email_At CHECK (
        email LIKE '%@%'
    ),
    CONSTRAINT CK_Users_Phone_Digits CHECK (
        phone IS NULL OR phone NOT LIKE '%[^0-9]%'
    )
);

```

3. Widoki

Wszystkie osoby przypisane do danej rezerwacji

```
CREATE VIEW vw_BookingPeopleList AS
SELECT
    b.booking_id,
    pl.person_id,
    pl.first_name,
    pl.last_name
FROM Bookings b
JOIN PersonList pl ON b.booking_id = pl.booking_id;
```

Dostępne oferty z liczbą wolnych miejsc

```
CREATE VIEW vw_AvailableOffers AS
SELECT
    o.offer_id,
    o.title,
    o.description,
    o.price,
    o.start_date,
    o.end_date,
    o.seats_limit - ISNULL(SUM(bd.num_of_people), 0) AS available_seats,
    c.country_name
FROM Offers o
JOIN Country c ON o.country_id = c.country_id
LEFT JOIN Bookings b ON o.offer_id = b.offer_id
LEFT JOIN BookingDetail bd ON b.booking_id = bd.booking_id
    AND bd.status IN ('confirmed', 'pending')
WHERE o.available = 1
    AND o.start_date > GETDATE()
GROUP BY o.offer_id, o.title, o.description, o.price,
    o.start_date, o.end_date, o.seats_limit, c.country_name;
```

Wszystkie osoby przypisane do danej atrakcji w ramach rezerwacji

```
CREATE VIEW vw_AttractionParticipants AS
SELECT
    a.attraction_id,
    a.title AS attraction_title,
    pl.person_id,
    pl.first_name,
    pl.last_name,
    o.title AS offer_title,
```

```

    b.booking_id
FROM PersonAttractions pa
JOIN PersonList pl ON pa.person_id = pl.person_id
JOIN Bookings b ON pl.booking_id = b.booking_id
JOIN OfferAttractions oa ON pa.booking_id = oa.offer_id
JOIN Offers o ON oa.offer_id = o.offer_id
JOIN Attractions a ON oa.attraction_id = a.attraction_id;

```

Raport rezerwacji z informacją o statusie płatności

```

ALTER VIEW vw_BookingReport AS
SELECT
    o.booking_id,
    o.customer_name,
    o.offer_title,
    o.booking_date,
    o.start_date,
    o.end_date,
    o.status,
    o.num_of_people,
    p.total_price_due,
    p.total_paid,
    p.total_price_due - p.total_paid AS owing,
    CASE
        WHEN p.is_fully_paid = 1 THEN 'paid'
        ELSE 'unpaid'
    END AS payment_status
FROM vw_OrderedOffers o
JOIN vw_BookingPaymentStatus_WithAttractions p ON o.booking_id = p.booking_id;

```

Podgląd które zamówienia zostały opłacone

```

CREATE FUNCTION fn_GetBookingPaymentStatus (
    @booking_id INT
)
RETURNS TABLE
AS
RETURN
(
    WITH AttractionCosts AS (
        SELECT
            ba.booking_id,
            SUM(a.price * ba.seats_reserved) AS total_attraction_cost
        FROM BookingAttractions ba
        JOIN OfferAttractions oa ON ba.offer_attraction_id = oa.offer_attraction_id
        JOIN Attractions a ON oa.attraction_id = a.attraction_id
        WHERE ba.booking_id = @booking_id
        GROUP BY ba.booking_id
    )

```

```

),
PaymentsTotal AS (
    SELECT
        booking_id,
        SUM(amount) AS total_paid
    FROM Payments
    WHERE booking_id = @booking_id
    GROUP BY booking_id
)
SELECT
    b.booking_id,
    -- Cena całkowita: cena oferty × liczba osób + suma atrakcji
    o.price * bd.num_of_people + ISNULL(ac.total_attraction_cost, 0) AS total_price_due,
    ISNULL(pt.total_paid, 0) AS total_paid,
    CASE
        WHEN ISNULL(pt.total_paid, 0) >= o.price * bd.num_of_people +
ISNULL(ac.total_attraction_cost, 0)
        THEN CAST(1 AS bit)
        ELSE CAST(0 AS bit)
    END AS is_fully_paid
FROM Bookings b
JOIN Offers o ON b.offer_id = o.offer_id
JOIN BookingDetail bd ON b.booking_id = bd.booking_id
LEFT JOIN AttractionCosts ac ON b.booking_id = ac.booking_id
LEFT JOIN PaymentsTotal pt ON b.booking_id = pt.booking_id
WHERE b.booking_id = @booking_id
);

```

Dostępność miejsc na poszczególnych atrakcjach w ofercie

```

CREATE VIEW vw_OfferAttractionsAvailability AS
SELECT
    oa.offer_id,
    o.title AS offer_title,
    a.attraction_id,
    a.title AS attraction_title,
    a.seats_limit - ISNULL(SUM(ba.seats_reserved), 0) AS available_seats
FROM OfferAttractions oa
JOIN Offers o ON oa.offer_id = o.offer_id
JOIN Attractions a ON oa.attraction_id = a.attraction_id
LEFT JOIN BookingAttractions ba ON oa.offer_attraction_id = ba.offer_attraction_id
GROUP BY oa.offer_id, o.title, a.attraction_id, a.title, a.seats_limit;

```

Szczegóły rezerwacji z danymi klienta i statusem

```
CREATE VIEW vw_OrderedOffers AS
SELECT
    b.booking_id,
    u.user_id,
    u.first_name + ' ' + u.last_name AS customer_name,
    o.offer_id,
    o.title AS offer_title,
    b.booking_date,
    b.start_date,
    b.end_date,
    bd.status,
    bd.num_of_people
FROM Bookings b
JOIN Users u ON b.user_id = u.user_id
JOIN Offers o ON b.offer_id = o.offer_id
JOIN BookingDetail bd ON b.booking_id = bd.booking_id;
```

Szczegóły płatności wraz z informacją o kliencie i ofercie

```
CREATE VIEW vw_PaymentsReport AS
SELECT
    b.booking_id, p.payment_id,
    p.payment_date,
    p.amount,
    p.payment_method,
    u.first_name + ' ' + u.last_name AS customer_name,
    o.title AS offer_title
FROM Payments p
JOIN Bookings b ON p.booking_id = b.booking_id
JOIN Users u ON b.user_id = u.user_id
JOIN Offers o ON b.offer_id = o.offer_id;
```

4. Procedury

Dodanie nowej oferty

```
CREATE PROCEDURE sp_AddOffer
    @Title NVARCHAR(255),
    @Description NVARCHAR(MAX),
    @Price MONEY,
    @StartDate DATE,
    @EndDate DATE,
    @SeatsLimit INT,
    @CountryID INT,
    @OfferID INT OUTPUT
AS
BEGIN
```

```
SET NOCOUNT ON;
```

```
-- Wstawienie nowej oferty
```

```
INSERT INTO Offers (
```

```
    title,
```

```
    description,
```

```
    price,
```

```
    start_date,
```

```
    end_date,
```

```
    seats_limit,
```

```
    available,
```

```
    country_id
```

```
) VALUES (
```

```
    @Title,
```

```
    @Description,
```

```
    @Price,
```

```
    @StartDate,
```

```
    @EndDate,
```

```
    @SeatsLimit,
```

```
    1, -- domyślnie dostępna
```

```
    @CountryID
```

```
);
```

```
-- Pobranie ID nowo dodanej oferty
```

```
SET @OfferID = SCOPE_IDENTITY();
```

```
END;
```

Przykład wywołania procedury:

```
DECLARE @NewOfferID INT;
```

```
EXEC sp_AddOffer
```

```
    @Title = N'Wakacje w Chorwacji',
```

```
    @Description = 'Zwiedzanie wysp śródziemnomorskich.',
```

```
    @Price = 3149.99,
```

```
    @StartDate = '2025-07-01',
```

```
    @EndDate = '2025-07-08',
```

```
    @SeatsLimit = 15,
```

```
    @CountryID = 7,
```

```
    @OfferID = @NewOfferID OUTPUT;
```

```
SELECT @NewOfferID AS NewOfferID;
```

Dodawanie Rezerwacji

```
CREATE PROCEDURE AddBooking
```

```
    @offer_id INT,
```

```
    @user_id INT,
```

```
    @num_of_people INT
```

```
AS
```

```
BEGIN
```

```
    SET NOCOUNT ON;
```

```
    BEGIN TRY
```

```
        BEGIN TRANSACTION;
```

```
        DECLARE @available_seats INT;
```

```
        DECLARE @start_date DATE;
```

```
        DECLARE @end_date DATE;
```

```
        -- Pobierz dostępne miejsca i daty oferty
```

```
        SELECT
```

```
            @available_seats = ao.available_seats,
```

```
            @start_date = o.start_date,
```

```
            @end_date = o.end_date
```

```
        FROM vw_AvailableOffers ao
```

```
        JOIN Offers o ON ao.offer_id = o.offer_id
```

```
        WHERE ao.offer_id = @offer_id;
```

```
        -- Walidacja dostępności miejsc
```

```
        IF @available_seats < @num_of_people
```

```
THROW 50001, 'Not enough available seats', 1;
```

```
-- Dodaj rezerwację
```

```
INSERT INTO Bookings (offer_id, user_id, start_date, end_date)
```

```
VALUES (@offer_id, @user_id, @start_date, @end_date);
```

```
DECLARE @new_booking_id INT = SCOPE_IDENTITY();
```

```
-- Dodaj szczegóły rezerwacji
```

```
INSERT INTO BookingDetail (booking_id, status, num_of_people)
```

```
VALUES (@new_booking_id, 'pending', @num_of_people);
```

```
COMMIT;
```

```
END TRY
```

```
BEGIN CATCH
```

```
ROLLBACK;
```

```
THROW;
```

```
END CATCH
```

```
END;
```

Przykładowa rezerwacja:

```
EXEC AddBooking
```

```
@offer_id = 18,
```

```
@user_id = 21,
```

```
@num_of_people = 5;
```

Dodawanie płatności do zamówienia

```
CREATE PROCEDURE Booking_AddPayment
    @BookingId INT,
    @Amount MONEY,
    @PaymentMethod NVARCHAR(50)
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @TotalPriceDue MONEY = 0;
    DECLARE @TotalPaid MONEY = 0;

    SELECT
        @TotalPriceDue = ISNULL(total_price_due, 0),
        @TotalPaid = ISNULL(total_paid, 0)
    FROM vw_BookingPaymentStatus
    WHERE booking_id = @BookingId;

    IF @TotalPriceDue = 0
    BEGIN
        THROW 50001, 'Booking not found or no due amount.', 1;
    END

    IF (@TotalPaid + @Amount) > @TotalPriceDue
    BEGIN
        THROW 50002, 'Payment exceeds total amount due.', 1;
    END

    INSERT INTO Payments (booking_id, amount, payment_method)
    VALUES (@BookingId, @Amount, @PaymentMethod);
END;

EXEC Booking_AddPayment
    @BookingId 1,
    @Amount 2000,
    @PaymentMethod 'bank transfer'
```

Rezerwowanie atrakcji

```
CREATE PROCEDURE BookAttractionForBooking
    @booking_id INT,
    @offer_attraction_id INT,
    @seats_reserved INT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION;
```

```
DECLARE @attr_id INT, @limit INT, @sum_reserved INT, @available INT,  
@booking_num_of_people INT;
```

```
-- Pobranie informacji o atrakcji powiązanej z ofertą
```

```
SELECT
```

```
    @attr_id = oa.attraction_id,
```

```
    @limit = a.seats_limit
```

```
FROM OfferAttractions oa
```

```
JOIN Attractions a ON oa.attraction_id = a.attraction_id
```

```
WHERE oa.offer_attraction_id = @offer_attraction_id;
```

```
IF @attr_id IS NULL
```

```
BEGIN
```

```
    RAISERROR('Niepoprawny offer_attraction_id', 16, 1);
```

```
    ROLLBACK TRANSACTION;
```

```
    RETURN;
```

```
END;
```

```
-- Pobranie liczby osób przypisanych do rezerwacji
```

```
SELECT @booking_num_of_people = num_of_people
```

```
FROM BookingDetail
```

```
WHERE booking_id = @booking_id;
```

```
IF @booking_num_of_people IS NULL
```

```
BEGIN
```

```
    RAISERROR('Nie znaleziono rezerwacji o ID %d', 16, 1, @booking_id);
```

```
    ROLLBACK TRANSACTION;
```

```
    RETURN;
```

```
END;
```

```
-- Sprawdzenie, czy liczba rezerwowanych miejsc nie przekracza liczby uczestników
```

```
IF @seats_reserved > @booking_num_of_people
```

```
BEGIN
```

```
    RAISERROR('Rezerwacja zawiera tylko %d uczestników, próbujesz przypisać %d miejsc.',
```

```
        16, 1, @booking_num_of_people, @seats_reserved);
```

```
    ROLLBACK TRANSACTION;
```

```
    RETURN;
```

```
END;
```

```
-- Sprawdzenie sumy dotychczas zarezerwowanych miejsc dla tej oferty
```

```
SELECT @sum_reserved = SUM(seats_reserved)
```

```
FROM BookingAttractions
```

```
WHERE offer_attraction_id = @offer_attraction_id;
```

```
SET @sum_reserved = ISNULL(@sum_reserved, 0);
```

```
SET @available = @limit - @sum_reserved;
```

```
-- Sprawdzenie czy jest wystarczająco wolnych miejsc
```

```
IF @seats_reserved > @available
```

```
BEGIN
```

```

        RAISERROR('Zbyt mało dostępnych miejsc (dostępnych %d, próbujesz zarezerwować %d)',
            16, 1, @available, @seats_reserved);
    ROLLBACK TRANSACTION;
    RETURN;
END;

-- Dodanie rezerwacji do BookingAttractions
INSERT INTO BookingAttractions (
    booking_id, offer_attraction_id, seats_reserved
) VALUES (
    @booking_id, @offer_attraction_id, @seats_reserved
);

COMMIT TRANSACTION;
END;

```

Przykład

EXEC BookAttractionForBooking

```

@booking_id = 31,
@offer_attraction_id = 11,
@seats_reserved = 2;

```

Usunięcie całej oferty

CREATE PROCEDURE CancelEntireOffer

```

    @offer_id INT

```

AS

BEGIN

```

    SET NOCOUNT ON;

```

```

    BEGIN TRANSACTION;

```

```
DELETE BA  
  
FROM BookingAttractions BA  
  
JOIN OfferAttractions OA ON BA.offer_attraction_id = OA.offer_attraction_id  
  
WHERE OA.offer_id = @offer_id;
```

```
DELETE FROM OfferAttractions  
  
WHERE offer_id = @offer_id;
```

```
DELETE FROM Offers  
  
WHERE offer_id = @offer_id;
```

```
COMMIT;  
  
END;
```

Przykład

```
EXEC CancelEntireOffer  
  
@offer_id = 1;
```

Przeniesienie przestarzałych danych do historii

```
CREATE PROCEDURE MoveExpiredBookingsToHistory  
  
AS  
  
BEGIN  
  
    SET NOCOUNT ON;  
  
    BEGIN TRANSACTION;  
  
  
  
  
  
  
  
  
    DECLARE @Now DATE = CAST(GETDATE() AS DATE);
```

```
DECLARE @FinishedBookings TABLE (booking_id INT);
```

```
INSERT INTO @FinishedBookings (booking_id)
```

```
SELECT booking_id
```

```
FROM u_mzes.dbo.Bookings
```

```
WHERE end_date < @Now;
```

```
INSERT INTO u_mzes.dbo.BookingHistory (
```

```
    original_booking_id, offer_id, user_id, booking_date,
```

```
    start_date, end_date, status, payment_status, num_of_people, archive_date
```

```
)
```

```
SELECT
```

```
    b.booking_id,
```

```
    b.offer_id,
```

```
    b.user_id,
```

```
    b.booking_date,
```

```
    b.start_date,
```

```
    b.end_date,
```

```
    bd.status,
```

```
CASE
```

```
    WHEN ISNULL(SUM(p.amount), 0) >= o.price * bd.num_of_people THEN 1
```

```
    ELSE 0
```

```
END AS payment_status,
```

```
    bd.num_of_people,
```

```
    GETDATE()
```

```
FROM u_mzes.dbo.Bookings b
```

```

JOIN u_mzes.dbo.BookingDetail bd ON b.booking_id = bd.booking_id
JOIN u_mzes.dbo.Offers o ON b.offer_id = o.offer_id
LEFT JOIN u_mzes.dbo.Payments p ON b.booking_id = p.booking_id
WHERE b.booking_id IN (SELECT booking_id FROM @FinishedBookings)
GROUP BY b.booking_id, b.offer_id, b.user_id, b.booking_date,
         b.start_date, b.end_date, bd.status, bd.num_of_people, o.price;

```

```

INSERT INTO u_mzes.dbo.PersonHistoryList (history_id, first_name, last_name)

```

```

SELECT

```

```

    bh.history_id,

```

```

    pl.first_name,

```

```

    pl.last_name

```

```

FROM u_mzes.dbo.PersonList pl

```

```

JOIN @FinishedBookings fb ON pl.booking_id = fb.booking_id

```

```

JOIN u_mzes.dbo.BookingHistory bh ON bh.original_booking_id = pl.booking_id;

```

```

INSERT INTO u_mzes.dbo.PersonHistoryAttractions (person_history_id, attraction_id)

```

```

SELECT

```

```

    phl.person_history_id,

```

```

    oa.attraction_id

```

```

FROM u_mzes.dbo.PersonAttractions pa

```

```

JOIN u_mzes.dbo.PersonList pl ON pa.person_id = pl.person_id

```

```

JOIN @FinishedBookings fb ON pl.booking_id = fb.booking_id

```

```

JOIN u_mzes.dbo.BookingHistory bh ON bh.original_booking_id = pl.booking_id

```

```

JOIN u_mzes.dbo.PersonHistoryList phl ON phl.history_id = bh.history_id

```

```

JOIN u_mzes.dbo.BookingAttractions ba ON pa.booking_attraction_id =
ba.booking_attraction_id

```

```

JOIN u_mzes.dbo.OfferAttractions oa ON ba.offer_attraction_id = oa.offer_attraction_id;

```

```
DELETE FROM u_mzes.dbo.PersonAttractions
WHERE person_id IN (
    SELECT person_id FROM u_mzes.dbo.PersonList
    WHERE booking_id IN (SELECT booking_id FROM @FinishedBookings)
);
```

```
DELETE FROM u_mzes.dbo.PersonList
WHERE booking_id IN (SELECT booking_id FROM @FinishedBookings);
```

```
DELETE FROM u_mzes.dbo.BookingAttractions
WHERE booking_id IN (SELECT booking_id FROM @FinishedBookings);
```

```
DELETE FROM u_mzes.dbo.BookingDetail
WHERE booking_id IN (SELECT booking_id FROM @FinishedBookings);
```

```
DELETE FROM u_mzes.dbo.Bookings
WHERE booking_id IN (SELECT booking_id FROM @FinishedBookings);
```

```
COMMIT
```

```
END;
```

Dodanie osoby do Booking

```
CREATE PROCEDURE AddPersonToBooking
    @booking_id INT,
    @first_name NVARCHAR(100),
    @last_name NVARCHAR(100)
AS
BEGIN
    SET NOCOUNT ON;
```

```
IF NOT EXISTS (SELECT 1 FROM Bookings WHERE booking_id = @booking_id)
BEGIN
    RAISERROR('Booking not found.', 16, 1);
    RETURN;
END
```

```
DECLARE @max_people INT;
SELECT @max_people = bd.num_of_people
FROM BookingDetail bd
WHERE bd.booking_id = @booking_id;
```

```
IF @max_people IS NULL
BEGIN
    RAISERROR('Booking detail not found.', 16, 1);
    RETURN;
END
```

```
DECLARE @current_people INT;
SELECT @current_people = COUNT(*) FROM PersonList WHERE booking_id = @booking_id;
```

```
IF @current_people >= @max_people
BEGIN
    RAISERROR('Cannot add person: booking seats limit exceeded.', 16, 1);
    RETURN;
END
```

```
INSERT INTO PersonList (booking_id, first_name, last_name)
VALUES (@booking_id, @first_name, @last_name);
END;
```

Przykład

EXEC AddPersonToBooking

@booking_id = 1,

@first_name = 'Adam',

@last_name = 'Nowak';

Dodanie osoby do atrakcji

```
CREATE PROCEDURE AddPersonToAttraction
    @person_id INT,
```

```

    @booking_id INT,
    @attraction_id INT
AS
BEGIN
    SET NOCOUNT ON;

    IF NOT EXISTS (
        SELECT 1 FROM PersonList WHERE person_id = @person_id AND booking_id =
@booking_id
    )
    BEGIN
        RAISERROR('Person does not belong to the specified booking.', 16, 1);
        RETURN;
    END

    DECLARE @offer_id INT;
    SELECT @offer_id = offer_id FROM Bookings WHERE booking_id = @booking_id;

    IF @offer_id IS NULL
    BEGIN
        RAISERROR('Offer not found.', 16, 1);
        RETURN;
    END

    DECLARE @offer_attraction_id INT;
    SELECT @offer_attraction_id = offer_attraction_id
    FROM OfferAttractions
    WHERE offer_id = @offer_id AND attraction_id = @attraction_id;

    IF @offer_attraction_id IS NULL
    BEGIN
        RAISERROR('Attraction is not associated with the offer for this booking.', 16, 1);
        RETURN;
    END

    DECLARE @booking_attraction_id INT;
    DECLARE @seats_reserved INT;

    SELECT
        @booking_attraction_id = booking_attraction_id,
        @seats_reserved = seats_reserved
    FROM BookingAttractions
    WHERE booking_id = @booking_id AND offer_attraction_id = @offer_attraction_id;

    IF @booking_attraction_id IS NULL
    BEGIN
        RAISERROR('BookingAttraction record not found for this booking and attraction.', 16, 1);
        RETURN;
    END

```

```

DECLARE @persons_count INT;
SELECT @persons_count = COUNT(*)
FROM PersonAttractions
WHERE booking_attraction_id = @booking_attraction_id;

IF @persons_count >= @seats_reserved
BEGIN
    RAISERROR('No available reserved seats left for this attraction in the booking.', 16, 1);
    RETURN;
END

INSERT INTO PersonAttractions (person_id, booking_id, booking_attraction_id)
VALUES (@person_id, @booking_id, @booking_attraction_id);

```

END

Przykład

```

EXEC AddPersonToAttraction
    @person_id = 1,
    @booking_id = 1,
    @attraction_id = 1;

```

Stworzenie atrakcji dla oferty

```
CREATE PROCEDURE dbo.AddAttractionToOffer
```

```

    @offer_id INT,

    @title NVARCHAR(255),

    @description NVARCHAR(MAX) = NULL,

    @seats_limit INT,

    @price MONEY = 0

```

AS

BEGIN

```
    SET NOCOUNT ON;
```

BEGIN TRY

```
    BEGIN TRANSACTION;
```

```

    -- Wstaw nową atrakcję

```

```
INSERT INTO Attractions (title, description, price, seats_limit)
VALUES (@title, @description, @price, @seats_limit);
```

```
DECLARE @newAttractionID INT = SCOPE_IDENTITY();
```

```
-- Wstaw powiązanie oferty z atrakcją
```

```
INSERT INTO OfferAttractions (offer_id, attraction_id)
VALUES (@offer_id, @newAttractionID);
```

```
COMMIT TRANSACTION;
```

```
END TRY
```

```
BEGIN CATCH
```

```
ROLLBACK TRANSACTION;
```

```
THROW;
```

```
END CATCH
```

```
END;
```

Przykład:

EXEC AddAttractionToOffer

```
@offer_id = 18,
```

```
@title = N'Rejs po morzu',
```

```
@description = N'Rejs statkiem Split-Dubrownik.',
```

```
@seats_limit = 20,
```

```
@price = 99.00;
```

Oznaczenie jako odwołane rezerwacji niedokończonych

```
CREATE PROCEDURE CancelUnpaidAndIncompleteBookings
AS
BEGIN
    SET NOCOUNT ON;
```

BEGIN TRANSACTION;

```
DECLARE @BookingsToCancel TABLE (  
    booking_id INT PRIMARY KEY,  
    missing_people BIT,  
    missing_attraction_assignments BIT  
);
```

```
INSERT INTO @BookingsToCancel (booking_id, missing_people,  
missing_attraction_assignments)  
SELECT  
    b.booking_id,  
    CASE WHEN COUNT(pl.person_id) < bd.num_of_people THEN 1 ELSE 0 END,  
    CASE WHEN EXISTS (  
        SELECT 1  
        FROM BookingAttractions ba  
        LEFT JOIN PersonAttractions pa ON ba.booking_attraction_id =  
pa.booking_attraction_id  
        WHERE ba.booking_id = b.booking_id  
        GROUP BY ba.booking_attraction_id, ba.seats_reserved  
        HAVING COUNT(pa.person_attraction_id) < ba.seats_reserved  
    ) THEN 1 ELSE 0 END  
FROM Bookings b  
JOIN BookingDetail bd ON b.booking_id = bd.booking_id  
JOIN vw_BookingPaymentStatus v ON b.booking_id = v.booking_id  
LEFT JOIN PersonList pl ON b.booking_id = pl.booking_id  
WHERE b.start_date <= DATEADD(DAY, 7, GETDATE())  
    AND b.start_date > GETDATE()  
    AND v.is_fully_paid = 0  
    AND bd.status <> 'cancelled'  
GROUP BY b.booking_id, bd.num_of_people  
HAVING COUNT(pl.person_id) < bd.num_of_people  
    OR EXISTS (  
        SELECT 1  
        FROM BookingAttractions ba  
        LEFT JOIN PersonAttractions pa ON ba.booking_attraction_id =  
pa.booking_attraction_id  
        WHERE ba.booking_id = b.booking_id  
        GROUP BY ba.booking_attraction_id, ba.seats_reserved  
        HAVING COUNT(pa.person_attraction_id) < ba.seats_reserved  
    );
```

```
UPDATE BookingDetail  
SET status = 'cancelled'  
WHERE booking_id IN (SELECT booking_id FROM @BookingsToCancel);
```

COMMIT TRANSACTION;

SELECT

```

bc.booking_id,
u.username,
u.first_name + ' ' + u.last_name AS customer_name,
o.title AS offer_title,
b.start_date,
v.total_price_due,
v.total_paid,
v.total_price_due - v.total_paid AS amount_due,
CASE
    WHEN bc.missing_people = 1 AND bc.missing_attraction_assignments = 1
        THEN 'Brak osób i przypisań atrakcji'
    WHEN bc.missing_people = 1
        THEN 'Brak osób'
    ELSE 'Brak przypisań atrakcji'
END AS cancellation_reason
FROM @BookingsToCancel bc
JOIN Bookings b ON bc.booking_id = b.booking_id
JOIN Users u ON b.user_id = u.user_id
JOIN Offers o ON b.offer_id = o.offer_id
JOIN vw_BookingPaymentStatus v ON b.booking_id = v.booking_id;

END;

```

Przykład

EXEC CancelUnpaidAndIncompleteBookings;

Dodawanie nowego użytkownika

```

CREATE PROCEDURE AddNewUser
    @username NVARCHAR(255),
    @password NVARCHAR(60),
    @first_name NVARCHAR(100),
    @last_name NVARCHAR(100),
    @email NVARCHAR(255),
    @phone NVARCHAR(20) = NULL,
    @birth_date DATE,
    @role_id INT = NULL,
    @is_individual BIT = 1
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        INSERT INTO Users (
            username, password, first_name, last_name,
            email, phone, birth_date, role_id, is_individual
        )
        VALUES (

```

```

        @username, @password, @first_name, @last_name,
        @email, @phone, @birth_date, @role_id, @is_individual
    );
END TRY
BEGIN CATCH
    -- Obsługa błędów — zwróć opis błędu
    DECLARE @ErrMsg NVARCHAR(4000) = ERROR_MESSAGE();
    DECLARE @ErrSeverity INT = ERROR_SEVERITY();
    DECLARE @ErrState INT = ERROR_STATE();
    RAISERROR(@ErrMsg, @ErrSeverity, @ErrState);
END CATCH
END;

```

Przykład wywołania procedury:

```

EXEC AddNewUser
    @username = 'jan.kowalski',
    @password = 'tajneHaslo123!',
    @first_name = 'Jan',
    @last_name = 'Kowalski',
    @email = 'jan.kowalski@example.com',
    @phone = '123456789',
    @birth_date = '1990-05-15',
    @role_id = 4,
    @is_individual = 1;

```

5. Triggery

Triggery blokujące możliwość edycji zamówień na tydzień przed

```

CREATE TRIGGER dbo.trg_BlockChangesAfterDeadline_{BookingDetail, BookingAttractions,
Payments, PersonList}
ON u_mjopek.dbo.{BookingDetail, BookingAttractions, Payments, PersonList}
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS (
        SELECT 1
        FROM inserted i
        INNER JOIN u_mjopek.dbo.Bookings b ON i.booking_id = b.booking_id
        WHERE b.start_date <= DATEADD(DAY, 7, CAST(GETDATE() AS DATE))
    )
    BEGIN
        THROW 50001, 'Zmiany w rezerwacji niedozwolone po tygodniu przed wyjazdem', 1;
        ROLLBACK TRANSACTION;
        RETURN;
    END

```

```

END
END

CREATE TRIGGER dbo.trg_BlockChangesAfterDeadline_Bookings
ON u_mjopek.dbo.Bookings
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS (
        SELECT 1
        FROM inserted i
        WHERE i.start_date <= DATEADD(DAY, 7, CAST(GETDATE() AS DATE))
    )
    BEGIN
        THROW 50001, 'Zmiany w rezerwacji niedozwolone po tygodniu przed wyjazdem', 1;
        ROLLBACK TRANSACTION;
        RETURN;
    END
END

```

Uwspólnienie nazw krajów

```

CREATE TRIGGER trg_FormatCountryNameTitleCase
ON dbo.Country
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE c
    SET
        c.country_name = UPPER(LEFT(i.country_name, 1)) +
        LOWER(SUBSTRING(i.country_name, 2, LEN(i.country_name)))
    FROM
        dbo.Country c
    JOIN
        inserted i ON c.country_id = i.country_id
    WHERE
        i.country_name IS NOT NULL AND LEN(i.country_name) > 0;

END;

```

Ustawiania statusu Booking przy dodaniu płatności

```

CREATE TRIGGER trg_UpdateBookingStatusAfterPayment

```

```

ON Payments
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE bd
    SET bd.status = 'confirmed'
    FROM u_mjopek.dbo.BookingDetail bd
    JOIN vw_BookingPaymentStatus bps ON bd.booking_id = bps.booking_id
    JOIN inserted i ON i.booking_id = bps.booking_id
    WHERE bps.is_fully_paid = 1
    AND bd.status <> 'confirmed';
END;

```

6. Funkcje

Wpłaty w danym okresie

```

CREATE FUNCTION fn_WplatyWDlaOkresu (
    @dataOd DATE,
    @dataDo DATE
)
RETURNS TABLE
AS
RETURN
(
    SELECT
        SUM(p.amount) AS total_amount,
        COUNT(*) AS liczba_transakcji
    FROM Payments p
    WHERE p.payment_date BETWEEN @dataOd AND @dataDo
);

```

Przykład:

```
SELECT * FROM fn_PaymentsInPeriod('2025-04-30', '2025-06-30');
```

Najczęściej wybierane destynacje(Top n)

```

CREATE FUNCTION fn_TopDestinations (@TopN INT)
RETURNS TABLE
AS
RETURN (
    SELECT TOP (@TopN)
        o.title,
        COUNT(*) AS total_bookings
    FROM Bookings b

```

```
JOIN Offers o ON b.offer_id = o.offer_id
GROUP BY o.title
ORDER BY total_bookings DESC
);
```

Przykład:

```
SELECT * FROM fn_TopDestinations(5);
```

Lista dochodów z konkretnej oferty

```
CREATE FUNCTION fn_RevenuePerOffer()
RETURNS TABLE
AS
RETURN (
    SELECT
        o.offer_id,
        o.title,
        SUM(p.amount) AS total_revenue
    FROM Offers o
    JOIN Bookings b ON b.offer_id = o.offer_id
    JOIN Payments p ON p.booking_id = b.booking_id
    GROUP BY o.offer_id, o.title
);
```

Przykład:

```
SELECT * FROM fn_RevenuePerOffer();
```

Dostępne atrakcje danej oferty

```
CREATE FUNCTION fn_GetAvailableAttractionsForOffer (
    @offer_id INT
)
RETURNS TABLE
AS
RETURN
(
    SELECT
        a.attraction_id,
        a.title AS attraction_title,
        a.description,
        a.price,
        ISNULL(a.seats_limit - SUM(ba.seats_reserved), a.seats_limit) AS available_seats
    FROM u_mjopek.dbo.OfferAttractions oa
    JOIN u_mjopek.dbo.Attractions a ON oa.attraction_id = a.attraction_id
    LEFT JOIN u_mjopek.dbo.BookingAttractions ba
        ON ba.offer_attraction_id = oa.offer_attraction_id
    WHERE oa.offer_id = @offer_id
);
```

```

GROUP BY a.attraction_id, a.title, a.description, a.price, a.seats_limit
HAVING ISNULL(a.seats_limit - SUM(ba.seats_reserved), a.seats_limit) > 0
)

```

Przykład:

```
SELECT * FROM fn_GetAvailableAttractionsForOffer(1);
```

Długość danego booking

```

CREATE FUNCTION fn_GetBookingLength(@booking_id INT)
RETURNS INT
AS
BEGIN
    DECLARE @length INT;

    SELECT @length = DATEDIFF(DAY, start_date, end_date) + 1
    FROM u_mjopek.dbo.Bookings
    WHERE booking_id = @booking_id;

    RETURN @length;
END;

```

Przykład:

```
SELECT dbo.fn_GetBookingLength(1) AS BookingLength;
```

Szczegóły płatności danego bookingu z atrakcjami

```

CREATE FUNCTION fn_GetBookingPaymentStatus (
    @booking_id INT
)
RETURNS TABLE
AS
RETURN
(
    SELECT
        b.booking_id,
        o.price * bd.num_of_people +
            ISNULL(SUM(a.price * ba.seats_reserved), 0) AS total_price_due,
        ISNULL(SUM(DISTINCT p.amount), 0) AS total_paid,
        CASE
            WHEN ISNULL(SUM(DISTINCT p.amount), 0) >=
                o.price * bd.num_of_people + ISNULL(SUM(a.price * ba.seats_reserved), 0)
            THEN CAST(1 AS bit)
            ELSE CAST(0 AS bit)
        END AS is_fully_paid
    FROM Bookings b

```

```

JOIN Offers o ON b.offer_id = o.offer_id
JOIN BookingDetail bd ON b.booking_id = bd.booking_id
LEFT JOIN Payments p ON b.booking_id = p.booking_id
LEFT JOIN BookingAttractions ba ON b.booking_id = ba.booking_id
LEFT JOIN OfferAttractions oa ON ba.offer_attraction_id = oa.offer_attraction_id
LEFT JOIN Attractions a ON oa.attraction_id = a.attraction_id
WHERE b.booking_id = @booking_id
GROUP BY b.booking_id, o.price, bd.num_of_people
);

```

Przykład:

```
SELECT * FROM dbo.fn_GetBookingPaymentStatus(123);
```

Jakie atrakcje zamówiliśmy

```

CREATE FUNCTION fn_GetBookedAttractionsForBooking (
    @booking_id INT
)
RETURNS TABLE
AS
RETURN
(
    SELECT
        a.attraction_id,
        a.title AS attraction_title,
        ba.seats_reserved
    FROM BookingAttractions ba
    JOIN OfferAttractions oa ON ba.offer_attraction_id = oa.offer_attraction_id
    JOIN Attractions a ON oa.attraction_id = a.attraction_id
    WHERE ba.booking_id = @booking_id
);

```

Przykład:

```
SELECT * FROM dbo.fn_GetBookedAttractionsForBooking(101);
```